PYTHON NETWORKING WORKBOOK

Python Socket Projects

Intermediate to advanced builds with the socket library protocols, proxies, servers and security tools

17	2	every
projects	levels	line tested

All code in this workbook was executed and verified on Python 3.8+ (standard library only)

Contents

1. Workbook Setup 3

2. Intermediate Projects 5

3. Advanced Projects 13

4. Testing and Engineering 27

1. Workbook Setup

This workbook is the practical companion to the *Python Socket Programming* guide. Each project is small enough to read in one sitting, runs with the standard library only, and teaches one idea that real network software depends on.

7	10	0
intermediate projects	advanced projects	third-party packages

Project map

#	Project	Level	Core concepts
1	TCP ping	Intermediate	`connect` timing, error handling
2	Wake-on-LAN	Intermediate	UDP broadcast, binary payloads
3	Key-value store	Intermediate	text protocol, threads, locks
4	HTTP file server	Intermediate	HTTP parsing, path-traversal defence
5	TCP port forwarder	Intermediate	bidirectional relay, half-close
6	Reliable UDP	Intermediate	ACKs, retransmission, sequence numbers
7	Honeypot logger	Intermediate	banners, structured logging
8	Non-blocking event loop	Advanced	output buffering, back-pressure
9	HMAC authentication	Advanced	challenge-response, replay defence
10	Hardened server	Advanced	rate limits, connection caps, timeouts
11	DNS client from scratch	Advanced	binary protocol, name compression
12	SOCKS5 proxy	Advanced	RFC 1928 handshake, tunnelling
13	Async pub/sub broker	Advanced	`asyncio`, slow consumers
14	TLS-terminating proxy	Advanced	`ssl`, handshake deadlines
15	Pre-fork server	Advanced	`SO_REUSEPORT`, multi-process
16	Packet header parser	Advanced	Ethernet, IPv4, TCP, UDP headers
17	Zero-copy I/O	Advanced	`recv_into`, `memoryview`, throughput

Conventions

- Each server keeps its settings in constants at the top (`HOST` , `PORT`) and starts from `main()` .
- Servers bind to `127.0.0.1` by default. Change it deliberately, never by accident.
- Several projects import the small `framing.py` toolbox below.
- Run a server in one terminal and test it from another with `nc` , `curl` or a short client script.

The toolbox: framing.py

Length-prefixed messages and JSON helpers from the first guide, saved as one module. Projects 9 and 12 import from it.

```python
# framing.py
import json
import struct

def recv_exact(sock, n):
    """Read exactly n bytes, or raise ConnectionError if the peer disconnects."""
    buf = bytearray()
    while len(buf) < n:
        chunk = sock.recv(n - len(buf))
        if not chunk:
            raise ConnectionError("peer closed connection")
        buf.extend(chunk)
    return bytes(buf)

def send_msg(sock, payload: bytes):
    sock.sendall(struct.pack("!I", len(payload)) + payload)

def recv_msg(sock, max_size=1_048_576):
    (length,) = struct.unpack("!I", recv_exact(sock, 4))
    if length > max_size:
        raise ValueError(f"message too large ({length} bytes)")
    return recv_exact(sock, length)

def send_json(sock, obj):
    send_msg(sock, json.dumps(obj).encode("utf-8"))

def recv_json(sock, max_size=1_048_576):
    return json.loads(recv_msg(sock, max_size).decode("utf-8"))
```

Run the examples: Save each listing under the file name shown in its first comment, for example `kvstore.py` , then run `python3 kvstore.py` .

2. Intermediate Projects

1. TCP ping

Concepts: connection timing, timeouts, graceful failure. ICMP ping needs privileges, but a TCP connect measures what really matters: can I reach this service, and how fast?

```python
# tcping.py
import socket
import statistics
import time

def tcping(host, port, count=5, timeout=2.0, interval=1.0):
    """Measure TCP connect latency, like ping but for a port."""
    times = []
    for seq in range(count):
        start = time.perf_counter()
        try:
            with socket.create_connection((host, port), timeout=timeout):
                ms = (time.perf_counter() - start) * 1000
                times.append(ms)
                print(f"seq={seq} {host}:{port} time={ms:.2f} ms")
        except OSError as e:
            print(f"seq={seq} {host}:{port} failed: {e}")
        time.sleep(interval)
    if times:
        loss = 100 * (count - len(times)) // count
        print(f"min/avg/max = {min(times):.2f}/{statistics.mean(times):.2f}/"
              f"{max(times):.2f} ms, loss {loss}%")
    return times

if __name__ == "__main__":
    tcping("127.0.0.1", 80)
```

Extend it: run it in a loop against several hosts, then print a live table.

2. Wake-on-LAN

Concepts: UDP broadcast, building a binary packet. The magic packet is six `0xFF` bytes followed by the target MAC address repeated sixteen times.

```python
# wol.py
import socket

def wake(mac, broadcast="255.255.255.255", port=9):
    """Send a Wake-on-LAN magic packet."""
    raw = bytes.fromhex(mac.replace(":", "").replace("-", ""))
    if len(raw) != 6:
        raise ValueError("a MAC address has 6 bytes")
    packet = b"\xff" * 6 + raw * 16              # 6 x FF, then the MAC 16 times
    with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as s:
        s.setsockopt(socket.SOL_SOCKET, socket.SO_BROADCAST, 1)
        s.sendto(packet, (broadcast, port))
    return packet

if __name__ == "__main__":
    wake("aa:bb:cc:dd:ee:ff")
```

Extend it: read MAC addresses from a small JSON file of named machines, so `wake("nas")` works.

3. Key-value store

Concepts: a line-based protocol, one thread per client, shared state guarded by a `Lock`. Test it with `nc 127.0.0.1 6380` and type `SET lang python`, `GET lang`, `KEYS`, `QUIT`.

```python
# kvstore.py
import socket
import threading

HOST, PORT = "127.0.0.1", 6380
store, lock = {}, threading.Lock()

def execute(line):
    parts = line.strip().split(" ", 2)
    cmd = parts[0].upper()
    with lock:                                  # the dict is shared by all threads
        if cmd == "SET" and len(parts) == 3:
            store[parts[1]] = parts[2]
            return "OK"
        if cmd == "GET" and len(parts) == 2:
            return store.get(parts[1], "(nil)")
        if cmd == "DEL" and len(parts) == 2:
            return "1" if store.pop(parts[1], None) is not None else "0"
        if cmd == "KEYS":
            return " ".join(sorted(store)) or "(empty)"
    return "ERR unknown command"

def handle(conn, addr):
    conn.settimeout(60)
    with conn, conn.makefile("rw", encoding="utf-8", newline="\n") as f:
        try:
            while line := f.readline(4097):      # cap the line length
                if len(line) > 4096:
                    f.write("ERR line too long\n")
                    f.flush()
                    break
                if line.strip().upper() == "QUIT":
                    break
                f.write(execute(line) + "\n")
                f.flush()
        except (OSError, ValueError):
            pass

def main():
    with socket.create_server((HOST, PORT)) as srv:
        print(f"kvstore listening on {HOST}:{PORT}")
        while True:
            conn, addr = srv.accept()
            threading.Thread(target=handle, args=(conn, addr), daemon=True).start()

if __name__ == "__main__":
    main()
```

Why the lock?: Several threads touch the same dictionary. Compound actions such as read-then-write are not atomic, so every access goes through `with lock`. Remove it and a stress test will eventually corrupt data.

Extend it: add `SETEX key seconds value` for expiry, then persistence by appending commands to a log file.

4. HTTP file server

Concepts: HTTP is plain text over TCP. The server reads the request head, validates it, serves a file from `ROOT` and closes the connection. Open `http://127.0.0.1:8080/` in a browser.

```
import mimetypes
import os
import socket
import threading
from urllib.parse import unquote, urlsplit

HOST, PORT = "127.0.0.1", 8080
ROOT = os.path.realpath(".")
STATUS = {200: "OK", 400: "Bad Request", 403: "Forbidden",
          404: "Not Found", 405: "Method Not Allowed"}

def respond(conn, code, body=b"", ctype="text/plain; charset=utf-8"):
    head = (f"HTTP/1.1 {code} {STATUS[code]}\r\n"
            f"Content-Type: {ctype}\r\n"
            f"Content-Length: {len(body)}\r\n"
            "Connection: close\r\n\r\n")
    conn.sendall(head.encode() + body)

def read_head(conn, limit=8192):
    data = b""
    while b"\r\n\r\n" not in data:
        chunk = conn.recv(1024)
        if not chunk or len(data) > limit:
            return None
        data += chunk
    return data.split(b"\r\n\r\n", 1)[0].decode("latin-1")
```

```
def serve_request(conn):
    head = read_head(conn)
    try:
        method, target, _version = head.split("\r\n")[0].split(" ", 2)
    except (ValueError, AttributeError):
        return respond(conn, 400, b"bad request")
    if method != "GET":
        return respond(conn, 405, b"method not allowed")
    rel = unquote(urlsplit(target).path).lstrip("/") or "index.html"
    if "\x00" in rel:
        return respond(conn, 400, b"bad request")
    path = os.path.realpath(os.path.join(ROOT, rel))
    if os.path.commonpath([path, ROOT]) != ROOT:          # path traversal guard
        return respond(conn, 403, b"forbidden")
    if not os.path.isfile(path):
        return respond(conn, 404, b"not found")
    with open(path, "rb") as f:
        body = f.read()
    respond(conn, 200, body, mimetypes.guess_type(path)[0] or "application/octet-stream")

def handle(conn):
    with conn:
        conn.settimeout(5)
        try:
            serve_request(conn)
        except OSError:
            pass

def main():
    with socket.create_server((HOST, PORT)) as srv:
        print(f"serving {ROOT} on http://{HOST}:{PORT}")
        while True:
            conn, _ = srv.accept()
            threading.Thread(target=handle, args=(conn,), daemon=True).start()

if __name__ == "__main__":
    main()
```

Path traversal: A request for `/../../etc/passwd` must never leave `ROOT`. The server resolves the real path (following symlinks) and checks it still lives under `ROOT` before opening anything. Never build a file path from network input without this check.

Extend it: add `Content-Length` limits, `HEAD` support, directory listings and `If-Modified-Since`.

5. TCP port forwarder

Concepts: a relay needs two threads, one per direction. When one side finishes sending, `shutdown(SHUT_WR)` passes the end-of-file on while the other direction keeps flowing. Start any TCP service on port 5000, run this, then connect to port 9000.

```python
# forwarder.py
import socket
import threading

LISTEN = ("127.0.0.1", 9000)
TARGET = ("127.0.0.1", 5000)

def pump(src, dst, label):
    total = 0
    try:
        while data := src.recv(65536):
            dst.sendall(data)
            total += len(data)
    except OSError:
        pass
    finally:
        try:
            dst.shutdown(socket.SHUT_WR)         # pass the EOF along
        except OSError:
            pass
    print(f"  {label}: {total} bytes")

def handle(client, addr):
    try:
        upstream = socket.create_connection(TARGET, timeout=5)
    except OSError as e:
        print("upstream failed:", e)
        client.close()
        return
    upstream.settimeout(None)
    print("forwarding", addr, "->", TARGET)
    t = threading.Thread(target=pump, args=(upstream, client, "down"), daemon=True)
    t.start()
    pump(client, upstream, "up")
    t.join()
    client.close()
    upstream.close()

def main():
    with socket.create_server(LISTEN) as srv:
        while True:
            client, addr = srv.accept()
            threading.Thread(target=handle, args=(client, addr), daemon=True).start()

if __name__ == "__main__":
    main()
```

Extend it: hex-dump the traffic, add an allow-list of client IPs, or round-robin across several targets.

6. Reliable UDP

Concepts: UDP loses packets, so reliability must be built on top. This is *stop-and-wait*: send one numbered message, wait for its ACK, resend on timeout. The receiver ACKs duplicates too, because the first ACK may have been lost. The `loss` argument simulates a bad network so you can watch the retries work.

```python
# rudp.py
import random
import socket
import struct

HDR = struct.Struct("!BI")                      # type, sequence number
DATA, ACK = 0, 1

def send_reliable(sock, addr, messages, timeout=0.2, retries=10):
    """Stop-and-wait: send one message, wait for its ACK, retry on timeout."""
    sock.settimeout(timeout)
    for seq, msg in enumerate(messages):
        packet = HDR.pack(DATA, seq) + msg
        for attempt in range(retries):
            sock.sendto(packet, addr)
            try:
                while True:
                    reply, _ = sock.recvfrom(64)
                    kind, ack = HDR.unpack(reply[:HDR.size])
                    if kind == ACK and ack == seq:
                        break                   # correct ACK
                break                           # message delivered
            except socket.timeout:
                continue                        # lost: send again
        else:
            raise TimeoutError(f"message {seq} was never acknowledged")

def receive_reliable(sock, count, loss=0.0):
    """Collect `count` in-order messages. `loss` simulates a lossy network."""
    expected, out = 0, []
    while expected < count:
        packet, addr = sock.recvfrom(2048)
        kind, seq = HDR.unpack(packet[:HDR.size])
        if kind != DATA or random.random() < loss:
            continue                            # pretend the packet vanished
        sock.sendto(HDR.pack(ACK, seq), addr)   # always ACK (even duplicates)
        if seq == expected:
            out.append(packet[HDR.size:])
            expected += 1
    return out
```

Try it on loopback: run `receive_reliable(sock, 10, loss=0.4)` in one thread and `send_reliable(sock, addr, messages)` in another. Ten messages still arrive complete and in order.

> **Limits of stop-and-wait:** Only one message is in flight, so throughput is poor on slow links. Real protocols use a sliding window. Also, if the final ACK is lost the sender cannot tell whether delivery succeeded (the two generals problem), which is why TCP has a closing handshake.

7. Honeypot logger

Concepts: a low-interaction honeypot listens on ports nobody should use, shows a believable banner, records what visitors send, and logs one JSON object per line. Defenders use it to learn what scans and credentials hit an exposed host.

```python
# honeypot.py
import json
import socket
import threading
import time

HOST = "127.0.0.1"                              # use "0.0.0.0" to face the network
PORTS = {2222: b"SSH-2.0-OpenSSH_8.9\r\n", 2121: b"220 FTP service ready\r\n"}
LOG = "honeypot.jsonl"
lock = threading.Lock()

def log(event):
    event["ts"] = time.strftime("%Y-%m-%dT%H:%M:%S")
    with lock, open(LOG, "a") as f:
        f.write(json.dumps(event) + "\n")

def handle(conn, addr, port, banner):
    with conn:
        conn.settimeout(5)
        try:
            conn.sendall(banner)
            data = conn.recv(256)               # whatever the visitor sends first
        except OSError:
            data = b""
    log({"src": addr[0], "sport": addr[1], "port": port,
         "data": data.decode("latin-1")})

def listen(port, banner):
    with socket.create_server((HOST, port)) as srv:
        while True:
            conn, addr = srv.accept()
            threading.Thread(target=handle, args=(conn, addr, port, banner),
                             daemon=True).start()

def main():
    for port, banner in PORTS.items():
        threading.Thread(target=listen, args=(port, banner), daemon=True).start()
    print("honeypot listening on", list(PORTS))
    threading.Event().wait()                    # run forever

if __name__ == "__main__":
    main()
```

A log line looks like this:

```
{"src": "127.0.0.1", "sport": 51642, "port": 2222, "data": "root:toor\r\n", "ts": "2026-10-18T12:00:00"}
```

> **Deploy responsibly:** Run honeypots only on systems and networks you own, isolated from anything valuable and with no real services behind them. Log files can contain attacker-supplied text, so treat them as untrusted data when you read them.

3. Advanced Projects

8. Non-blocking event loop with buffering

Concepts: the single-threaded `selectors` server from the first guide ignored one hard fact: `send()` may accept only part of your data. A real loop queues unsent bytes per connection, listens for `EVENT_WRITE` only while the queue is not empty, and stops reading when a client is too slow to keep up (back-pressure).

```
import selectors
import socket

HOST, PORT = "127.0.0.1", 5000
HIGH_WATER = 1 << 20                            # stop reading above 1 MiB queued
sel = selectors.DefaultSelector()

class Conn:
    def __init__(self, sock):
        self.sock = sock
        self.outbuf = bytearray()
        sel.register(sock, selectors.EVENT_READ, self)

    def handle(self, mask):
        if mask & selectors.EVENT_READ:
            self.on_read()
        if mask & selectors.EVENT_WRITE and self.sock.fileno() != -1:
            self.flush()

    def on_read(self):
        try:
            data = self.sock.recv(65536)
        except BlockingIOError:
            return
        except OSError:
            data = b""
        if not data:
            return self.close()
        self.outbuf += data.upper()             # "process" the request
        self.flush()

    def flush(self):
        try:
            sent = self.sock.send(self.outbuf)
            del self.outbuf[:sent]              # keep only what is still unsent
        except BlockingIOError:
            pass
        except OSError:
            return self.close()
        if not self.outbuf:
            events = selectors.EVENT_READ
        elif len(self.outbuf) > HIGH_WATER:
            events = selectors.EVENT_WRITE      # back-pressure: stop reading
        else:
            events = selectors.EVENT_READ | selectors.EVENT_WRITE
        sel.modify(self.sock, events, self)

    def close(self):
        sel.unregister(self.sock)
        self.sock.close()
```

```
def main():
    srv = socket.create_server((HOST, PORT))
    srv.setblocking(False)
    sel.register(srv, selectors.EVENT_READ, None)
    while True:
        for key, mask in sel.select():
            if key.data is None:                # the listening socket
                try:
                    conn, _ = srv.accept()
                except BlockingIOError:
                    continue
                conn.setblocking(False)
                Conn(conn)
            else:
                key.data.handle(mask)

if __name__ == "__main__":
    main()
```

What this proves: Pushing three megabytes through this server while the client reads at the same time returns every byte, uppercased and in order. Without the output buffer, data would be silently lost or the loop would block.

9. HMAC challenge-response authentication

Concepts: prove knowledge of a shared secret without ever sending it. The server issues a random nonce, the client returns `HMAC(secret, nonce)`. Because the nonce is new every time, a captured reply is useless later. `hmac.compare_digest` avoids timing leaks.

```
# auth.py
import hashlib
import hmac
import os
from framing import recv_msg, send_msg

SECRET = b"change-me-use-a-long-random-key"

def server_handshake(conn):
    """Challenge the client; return True only if it knows the shared secret."""
    nonce = os.urandom(16)                      # fresh every time: no replay
    send_msg(conn, nonce)
    reply = recv_msg(conn, max_size=64)
    expected = hmac.new(SECRET, nonce, hashlib.sha256).digest()
    ok = hmac.compare_digest(reply, expected)    # constant-time comparison
    send_msg(conn, b"OK" if ok else b"NO")
    return ok

def client_handshake(sock):
    nonce = recv_msg(sock, max_size=64)
    send_msg(sock, hmac.new(SECRET, nonce, hashlib.sha256).digest())
    return recv_msg(sock, max_size=8) == b"OK"
```

This is authentication, not encryption: An attacker on the path can still read the traffic and can relay the handshake to the real server. Run it inside TLS (project 14) and authenticate the server too. Also use a long random secret, not a word.

Choosing an authentication method

Method	Strength	Weakness	Use when
Shared-secret HMAC	Simple, secret never sent	Key distribution, no encryption	Devices talking to a server inside TLS
TLS client certificates	Strong, per-client revocation	Certificate management	Internal services, mutual TLS
Password plus salted hash (scrypt, Argon2)	Familiar to users	Needs rate limiting and TLS	Human logins
Signed tokens or session IDs	Cheap to verify	Theft means access, needs expiry	After an initial login

10. Hardened server

Concepts: every public server should limit connections, request rate, line length and idle time. This one caps simultaneous clients with a semaphore, rate-limits each IP with a *token bucket* (refills at `RATE` per second, bursts up to `BURST`), and drops silent connections after `IDLE` seconds.

```python
# hardened.py
import socket
import threading
import time
from collections import defaultdict

HOST, PORT = "127.0.0.1", 5000
MAX_CONNS, RATE, BURST, IDLE = 50, 5.0, 10, 30   # RATE = requests/second per IP

slots = threading.BoundedSemaphore(MAX_CONNS)
buckets = defaultdict(lambda: [float(BURST), time.monotonic()])   # ip -> [tokens, last]
lock = threading.Lock()

def allow(ip):
    """Token bucket: refill at RATE per second, spend one token per request."""
    with lock:
        tokens, last = buckets[ip]
        now = time.monotonic()
        tokens = min(BURST, tokens + (now - last) * RATE)
        if tokens < 1:
            buckets[ip] = [tokens, now]
            return False
        buckets[ip] = [tokens - 1, now]
        return True

def handle(conn, addr):
    try:
        conn.settimeout(IDLE)                    # drop idle or slow clients
        with conn, conn.makefile("rwb") as f:
            while line := f.readline(1025):
                if len(line) > 1024 and not line.endswith(b"\n"):
                    f.write(b"ERR line too long\n")
                    f.flush()
                    break
                if not allow(addr[0]):
                    f.write(b"ERR rate limited\n")
                else:
                    f.write(line.upper())
                f.flush()
    except OSError:
        pass
    finally:
        slots.release()

def main():
    with socket.create_server((HOST, PORT)) as srv:
        while True:
            conn, addr = srv.accept()
            if not slots.acquire(blocking=False):
                conn.close()                     # over capacity: refuse quickly
                continue
            threading.Thread(target=handle, args=(conn, addr), daemon=True).start()

if __name__ == "__main__":
    main()
```

Sending twenty lines instantly returns the first ten processed and the rest answered with `ERR rate limited`.

Extend it: expire old entries from `buckets` so memory stays bounded, and add a ban list for repeat offenders.

11. DNS client from scratch

Concepts: DNS is a compact binary protocol over UDP. You pack a 12-byte header and a question, then decode the answer, including *name compression*, where a two-byte pointer replaces a repeated name. The code checks the transaction ID and rejects pointer loops.

```
import random
import socket
import struct

def build_query(name, qtype=1):
    """Build a DNS query. qtype: 1 = A, 28 = AAAA, 5 = CNAME."""
    header = struct.pack("!HHHHHH", random.randrange(65536), 0x0100, 1, 0, 0, 0)
    qname = b"".join(bytes([len(p)]) + p.encode("ascii")
                     for p in name.rstrip(".").split(".")) + b"\x00"
    return header + qname + struct.pack("!HH", qtype, 1)       # class IN

def read_name(msg, off):
    """Decode a (possibly compressed) domain name; return (name, next_offset)."""
    labels, end, hops = [], None, 0
    while True:
        n = msg[off]
        if n == 0:
            off += 1
            break
        if n & 0xC0 == 0xC0:                    # compression pointer
            hops += 1
            if hops > 10:
                raise ValueError("compression loop")
            if end is None:
                end = off + 2
            off = ((n & 0x3F) << 8) | msg[off + 1]
            continue
        labels.append(msg[off + 1:off + 1 + n].decode("ascii"))
        off += 1 + n
    return ".".join(labels), (off if end is None else end)
```

```
def parse_response(msg):
    _id, flags, qd, an, _ns, _ar = struct.unpack("!HHHHHH", msg[:12])
    off, answers = 12, []
    for _ in range(qd):
        _, off = read_name(msg, off)
        off += 4                                  # skip QTYPE + QCLASS
    for _ in range(an):
        name, off = read_name(msg, off)
        rtype, _cls, ttl, rdlen = struct.unpack("!HHIH", msg[off:off + 10])
        off += 10
        rdata = msg[off:off + rdlen]
        if rtype == 1:
            answers.append((name, "A", ttl, socket.inet_ntoa(rdata)))
        elif rtype == 28:
            answers.append((name, "AAAA", ttl, socket.inet_ntop(socket.AF_INET6, rdata)))
        elif rtype == 5:
            answers.append((name, "CNAME", ttl, read_name(msg, off)[0]))
        off += rdlen
    return flags & 0xF, answers                   # (rcode, records)

def resolve(name, server=("8.8.8.8", 53), qtype=1, timeout=3):
    query = build_query(name, qtype)
    with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as s:
        s.settimeout(timeout)
        s.sendto(query, server)
        msg, _ = s.recvfrom(4096)
    if msg[:2] != query[:2]:                      # transaction ID must match
        raise ValueError("response does not match the query")
    return parse_response(msg)

if __name__ == "__main__":
    print(resolve("example.com"))
```

Try `resolve("example.com", qtype=28)` for IPv6 records. Tested against a local fake server that returns a CNAME and an A record using compression pointers.

> **Why random IDs matter:** The 16-bit transaction ID, plus a random source port, is what stops an off-path attacker from forging answers (cache poisoning). Production resolvers randomise both.

12. SOCKS5 proxy

Concepts: SOCKS5 (RFC 1928) is a tiny handshake that lets any TCP client reach any destination through a proxy: greet and choose a method, send a CONNECT request with an IPv4, IPv6 or domain target, receive a status reply, then relay bytes. Point `curl --socks5-hostname 127.0.0.1:1080 http://example.com` at it.

```python
import socket
import struct
import threading
from framing import recv_exact

HOST, PORT = "127.0.0.1", 1080

def relay(a, b):
    def pump(src, dst):
        try:
            while data := src.recv(65536):
                dst.sendall(data)
        except OSError:
            pass
        finally:
            try:
                dst.shutdown(socket.SHUT_WR)
            except OSError:
                pass
    t = threading.Thread(target=pump, args=(b, a), daemon=True)
    t.start()
    pump(a, b)
    t.join()
```

```
def reply(code):
    return b"\x05" + bytes([code]) + b"\x00\x01" + bytes(6)   # BND.ADDR/PORT unused

def handle(client):
    with client:
        client.settimeout(10)
        try:
            ver, n = recv_exact(client, 2)
            methods = recv_exact(client, n)
            if ver != 5 or 0 not in methods:             # we only offer "no auth"
                client.sendall(b"\x05\xff")
                return
            client.sendall(b"\x05\x00")
            _ver, cmd, _rsv, atyp = recv_exact(client, 4)
            if cmd != 1:                                 # only CONNECT
                return client.sendall(reply(7))
            if atyp == 1:
                host = socket.inet_ntoa(recv_exact(client, 4))
            elif atyp == 3:
                host = recv_exact(client, recv_exact(client, 1)[0]).decode()
            elif atyp == 4:
                host = socket.inet_ntop(socket.AF_INET6, recv_exact(client, 16))
            else:
                return client.sendall(reply(8))
            (port,) = struct.unpack("!H", recv_exact(client, 2))
            try:
                remote = socket.create_connection((host, port), timeout=10)
            except OSError:
                return client.sendall(reply(5))          # connection refused
            with remote:
                client.sendall(reply(0))
                client.settimeout(None)
                remote.settimeout(None)
                relay(client, remote)
        except OSError:
            pass
```

```
def socks5_connect(proxy, host, port):
    """Client side: open a connection to host:port through a SOCKS5 proxy."""
    s = socket.create_connection(proxy)
    s.sendall(b"\x05\x01\x00")
    if recv_exact(s, 2) != b"\x05\x00":
        raise ConnectionError("proxy refused the auth method")
    h = host.encode()
    s.sendall(b"\x05\x01\x00\x03" + bytes([len(h)]) + h + struct.pack("!H", port))
    rep = recv_exact(s, 10)
    if rep[1] != 0:
        raise ConnectionError(f"SOCKS5 error code {rep[1]}")
    return s

def main():
    with socket.create_server((HOST, PORT)) as srv:
        while True:
            client, _ = srv.accept()
            threading.Thread(target=handle, args=(client,), daemon=True).start()

if __name__ == "__main__":
    main()
```

Never expose an open proxy: Bind to localhost or add authentication. An open proxy is abused to hide attacks and, on a server, can reach internal addresses that outsiders should never touch (SSRF). Production code blocks private ranges and logs targets.

13. Asynchronous pub/sub broker

Concepts: a message broker in about forty lines of `asyncio`. Clients send JSON lines such as `{"op": "sub", "topic": "news"}` and `{"op": "pub", "topic": "news", "msg": "hello"}`. Every subscriber of that topic receives the message. A slow consumer whose buffer grows past 1 MB is disconnected so it cannot exhaust server memory.

```python
# pubsub.py
import asyncio
import json
from collections import defaultdict

HOST, PORT = "127.0.0.1", 7000
topics = defaultdict(set)                       # topic -> set of subscriber writers

async def handle(reader, writer):
    mine = set()
    try:
        while line := await reader.readline():
            try:
                cmd = json.loads(line)
                op, topic = cmd["op"], cmd["topic"]
            except (ValueError, KeyError, TypeError):
                writer.write(b'{"error": "bad command"}\n')
                await writer.drain()
                continue
            if op == "sub":
                topics[topic].add(writer)
                mine.add(topic)
            elif op == "pub":
                payload = (json.dumps({"topic": topic, "msg": cmd.get("msg")}) + "\n").encode()
                for w in list(topics[topic]):
                    if w.transport.get_write_buffer_size() > 1_000_000:
                        topics[topic].discard(w)         # slow consumer: cut it off
                        w.close()
                    else:
                        w.write(payload)
            await writer.drain()
    except (ConnectionError, ValueError):        # ValueError: line over the 64 KiB limit
        pass
    finally:
        for t in mine:
            topics[t].discard(writer)
        writer.close()

async def main():
    server = await asyncio.start_server(handle, HOST, PORT)
    async with server:
        await server.serve_forever()

if __name__ == "__main__":
    asyncio.run(main())
```

Try it with three terminals running `nc 127.0.0.1 7000` : two subscribers and one publisher.

14. TLS-terminating proxy

Concepts: put encryption in front of a service that speaks plaintext. The proxy performs the TLS handshake (with a deadline, so slow or hostile clients cannot hold it open), then relays decrypted bytes to the backend. Create `cert.pem` and `key.pem` with the `openssl` command from the first guide.

```python
# tlsproxy.py
import socket
import ssl
import threading

LISTEN, BACKEND = ("127.0.0.1", 8443), ("127.0.0.1", 5000)

def pump(src, dst):
    try:
        while data := src.recv(65536):
            dst.sendall(data)
    except OSError:
        pass
    finally:
        for s in (src, dst):                    # when one side ends, end both
            try:
                s.shutdown(socket.SHUT_RDWR)
            except OSError:
                pass

def handle(raw, ctx):
    try:
        raw.settimeout(5)                       # handshake deadline
        client = ctx.wrap_socket(raw, server_side=True)
    except OSError:                             # ssl.SSLError is an OSError
        raw.close()
        return
    try:
        backend = socket.create_connection(BACKEND, timeout=5)
    except OSError:
        client.close()
        return
    client.settimeout(None)
    backend.settimeout(None)
    t = threading.Thread(target=pump, args=(backend, client), daemon=True)
    t.start()
    pump(client, backend)
    t.join()
    client.close()
    backend.close()

def main():
    ctx = ssl.SSLContext(ssl.PROTOCOL_TLS_SERVER)
    ctx.minimum_version = ssl.TLSVersion.TLSv1_2
    ctx.load_cert_chain("cert.pem", "key.pem")
    with socket.create_server(LISTEN) as srv:
        while True:
            raw, _ = srv.accept()
            threading.Thread(target=handle, args=(raw, ctx), daemon=True).start()

if __name__ == "__main__":
    main()
```

The tests confirmed three things: a verified TLS client reaches the backend, a plaintext client is rejected without crashing the proxy, and the next TLS client still works.

Extend it: add client certificates (mutual TLS), SNI-based routing, and access logging.

15. Pre-fork server with SO_REUSEPORT

Concepts: Python threads share one interpreter lock, so CPU-bound servers need processes. With `SO_REUSEPORT`, several processes bind the same port and the kernel spreads incoming connections among them. Linux and BSD; use `os.fork`, so not available on Windows.

```python
# prefork.py
import os
import socket

HOST, PORT, WORKERS = "127.0.0.1", 5000, 4

def worker():
    srv = socket.socket()
    srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEPORT, 1)   # kernel balances accepts
    srv.bind((HOST, PORT))
    srv.listen(128)
    while True:
        conn, _ = srv.accept()
        with conn:
            conn.sendall(f"served by pid {os.getpid()}\n".encode())

def main():
    for _ in range(WORKERS):
        if os.fork() == 0:                      # child process
            worker()
            os._exit(0)
    for _ in range(WORKERS):                    # parent waits for the children
        os.wait()

if __name__ == "__main__":
    main()
```

Forty test connections were answered by all four worker PIDs.

Extend it: supervise workers and restart any that die, or hand connections to a `ProcessPoolExecutor`.

16. Packet header parser

Concepts: decode raw network frames with `struct`: Ethernet (14 bytes), IPv4 (length from the IHL field) and TCP or UDP. `parse_frame` works on any bytes, so you can test it without privileges. `sniff()` captures live frames on Linux with `AF_PACKET` and needs root.

```
# sniffer.py
import socket
import struct

TCP_FLAGS = ((0x02, "SYN"), (0x10, "ACK"), (0x01, "FIN"),
             (0x04, "RST"), (0x08, "PSH"), (0x20, "URG"))

def parse_frame(frame):
    """Decode Ethernet -> IPv4 -> TCP/UDP headers from raw bytes."""
    _dst, _src, eth_type = struct.unpack("!6s6sH", frame[:14])
    if eth_type != 0x0800:
        return None                              # not IPv4
    ip = frame[14:]
    ver_ihl, _tos, _length, _id, _frag, ttl, proto, _csum, s_ip, d_ip = \
        struct.unpack("!BBHHHBBH4s4s", ip[:20])
    info = {"src": socket.inet_ntoa(s_ip), "dst": socket.inet_ntoa(d_ip), "ttl": ttl}
    l4 = ip[(ver_ihl & 0x0F) * 4:]               # IHL is counted in 32-bit words
    if proto == 6 and len(l4) >= 14:
        sport, dport, seq, ack, off_flags = struct.unpack("!HHIIH", l4[:14])
        flags = [name for bit, name in TCP_FLAGS if off_flags & bit]
        info.update(proto="TCP", sport=sport, dport=dport, flags="|".join(flags))
    elif proto == 17 and len(l4) >= 8:
        sport, dport, length, _ = struct.unpack("!HHHH", l4[:8])
        info.update(proto="UDP", sport=sport, dport=dport, length=length)
    else:
        info["proto"] = proto
    return info

def sniff(count=10):
    """Linux only, needs root. Capture only on networks you are allowed to monitor."""
    with socket.socket(socket.AF_PACKET, socket.SOCK_RAW, socket.ntohs(0x0003)) as s:
        for _ in range(count):
            frame, _ = s.recvfrom(65535)
            info = parse_frame(frame)
            if info:
                print(info)

if __name__ == "__main__":
    sniff()
```

Output for a captured packet looks like `{'src': '10.0.0.5', 'dst': '93.184.216.34', 'ttl': 64, 'proto': 'TCP', 'sport': 51000, 'dport': 443, 'flags': 'SYN'}`.

> **Capture only what you are allowed to:** Sniff only networks and hosts you own or have written permission to monitor. For protocol-level work beyond reading headers, look at Wireshark and the Scapy library.

17. Zero-copy I/O and throughput

Concepts: `recv()` allocates a new `bytes` object on every call. `recv_into()` fills a buffer you already own, and a `memoryview` slices it without copying. The benchmark also shows how much write size matters.

```python
# perf.py
import socket
import threading
import time

def sink(srv, result):
    conn, _ = srv.accept()
    buf, total = bytearray(65536), 0
    with conn:
        while n := conn.recv_into(buf):          # reuse one buffer: no allocations
            total += n
    result.append(total)

def recv_into_exact(sock, n):
    """Zero-copy version of recv_exact: fill one preallocated buffer."""
    buf = bytearray(n)
    view, got = memoryview(buf), 0
    while got < n:
        k = sock.recv_into(view[got:])
        if k == 0:
            raise ConnectionError("peer closed connection")
        got += k
    return buf

def throughput(chunk, total_mb=50):
    srv = socket.create_server(("127.0.0.1", 0))
    port, result = srv.getsockname()[1], []
    t = threading.Thread(target=sink, args=(srv, result))
    t.start()
    data, target, sent = memoryview(bytes(chunk)), total_mb << 20, 0
    start = time.perf_counter()
    with socket.create_connection(("127.0.0.1", port)) as s:
        while sent < target:
            s.sendall(data)
            sent += chunk
    t.join()
    srv.close()
    return sent / 1e6 / (time.perf_counter() - start)

if __name__ == "__main__":
    for chunk in (256, 4096, 65536):
        print(f"{chunk:>6}-byte writes: {throughput(chunk):7.0f} MB/s")
```

Sample output from the test machine (yours will differ):

Write size	Throughput
256 bytes	about 240 MB/s
4 KiB	about 2,100 MB/s
64 KiB	about 3,300 MB/s

> **Lesson:** Many tiny `send()` calls are expensive, because each one is a system call. Batch small writes into larger buffers before sending.

4. Testing and Engineering

Protocol design checklist

Question	Good answer
Where does a message end?	Length prefix or delimiter, never "one `recv` equals one message"
What is the maximum size?	A hard cap, checked before allocating memory
What if the peer is silent?	Timeouts on every socket and a heartbeat for long sessions
How are versions handled?	A version byte in the header from day one
How are errors reported?	A defined error reply, not a dropped connection
Who is the peer?	Authentication, ideally inside TLS
What if the peer is hostile?	Rate limits, connection caps, strict input validation
How do you debug it?	Text protocols or a documented binary layout, plus logging

Concurrency trouble spots

Symptom	Likely cause	Fix
Random corrupted shared data	Missing lock around read-modify-write	Guard shared state or use queues
Server freezes under load	Blocking call in an event loop	Use non-blocking I/O or move work to threads
Memory grows without limit	Unbounded buffers or per-IP tables	Cap sizes, expire old entries
Data lost on large transfers	Ignoring partial `send()`	`sendall()` or an output buffer
Handles leak over days	Sockets not closed on error paths	`with` blocks and `finally`
One slow client slows all	Shared thread or no back-pressure	Timeouts, per-client buffers, disconnect slow peers
Server dies on a bad packet	Uncaught exception in a handler	Catch `OSError`, `ValueError`, and log

Testing network code with unittest

Make network tests boring: bind to port `0` (the OS picks a free port), set a timeout on every socket, and run the real server code in a background thread. This suite tests project 3, including 20 concurrent clients. Run it with `python3 -m unittest test_kvstore -v`.

```
# test_kvstore.py
import socket
import threading
import unittest
import kvstore

class KVStoreTest(unittest.TestCase):
    @classmethod
    def setUpClass(cls):
        cls.srv = socket.create_server(("127.0.0.1", 0))      # port 0: never clashes
        cls.addr = cls.srv.getsockname()
        threading.Thread(target=cls.accept_loop, daemon=True).start()

    @classmethod
    def accept_loop(cls):
        while True:
            try:
                conn, addr = cls.srv.accept()
            except OSError:
                return                                        # server socket closed
            threading.Thread(target=kvstore.handle, args=(conn, addr), daemon=True).start()

    def setUp(self):
        kvstore.store.clear()
        self.sock = socket.create_connection(self.addr, timeout=2)   # a test must never hang
        self.file = self.sock.makefile("rw", newline="\n")
        self.addCleanup(self.sock.close)

    def ask(self, line):
        self.file.write(line + "\n")
        self.file.flush()
        return self.file.readline().strip()

    def test_set_then_get(self):
        self.assertEqual(self.ask("SET lang python"), "OK")
        self.assertEqual(self.ask("GET lang"), "python")

    def test_errors(self):
        self.assertEqual(self.ask("GET nothing"), "(nil)")
        self.assertTrue(self.ask("HACK").startswith("ERR"))

    def test_concurrent_clients(self):
        def worker(i):
            with socket.create_connection(self.addr, timeout=2) as s:
                f = s.makefile("rw", newline="\n")
                f.write(f"SET k{i} {i}\n")
                f.flush()
                f.readline()
        threads = [threading.Thread(target=worker, args=(i,)) for i in range(20)]
        for t in threads:
            t.start()
        for t in threads:
            t.join()
        self.assertEqual(len(self.ask("KEYS").split()), 20)
```

Where to go next

Build next

1. A mini Redis with persistence and pub/sub
2. A chat server with TLS, users and rooms
3. An HTTP/1.1 server with keep-alive and routing
4. A load balancer with health checks
5. A VPN-style tunnel with authenticated encryption

Study next

1. Sliding-window protocols and congestion control
2. `asyncio` protocols, transports and streams
3. The RFCs: 793 (TCP), 768 (UDP), 1928 (SOCKS5), 1035 (DNS)
4. Wireshark captures of your own programs
5. Fuzzing your parsers with random and malformed input

> **Final advice:** Treat every byte from a socket as untrusted until it has been length-checked, validated and parsed. That single habit prevents most network vulnerabilities.